

PERSONAL ENGINEERING PROJECT

MECHATRONICS / ROBOTICS / EMBEDDED CONTROL

PROJECT CASE STUDY

AUTONOMOUS BALL COLLECTION ROBOT

Design, control and prototype development

PERSONAL MECHATRONICS PROJECT

Autonomous ball collection, seesaw traversal and delivery system

CREATED BY

Mevan Shandika Perera

PROJECT FOCUS

Arduino control, actuation and prototyping

COMPLETED

13 June 2025

Executive Summary

Autonomous collection, transport and delivery prototype

This project explores a compact autonomous robot designed to collect three balls of different sizes, transport them securely across a pivoting seesaw, and release them at a destination. The prototype combines a four-wheel drive platform, two servo-actuated retaining arms, and an Arduino Uno running a calibrated time-based control sequence.

CONTROL

Arduino Uno

Sequential motor and servo commands using calibrated timing.

MOBILITY

Four-wheel drive

Four 12 V geared DC motors provide movement and seesaw traversal.

HANDLING

Dual servo arms

Custom grooves and servo arms secure three different ball sizes.

Development outcome

Testing demonstrated successful collection, controlled navigation, and stable seesaw crossing. The table-tennis ball was released successfully; the heavier tennis and racquet balls required more release momentum. The result validated the core mechanical and control architecture while identifying clear opportunities for sensing, voltage regulation, and an assisted release mechanism.

KEY DESIGN DECISION

Separate power supplies were used for the drive motors and servo system to reduce voltage drop and improve repeatability during combined operation.

Contents

1	Introduction	4
2	Initial Design	5
2.1	Initial timelines	6
3	Final Design	7
3.1	Mechanical structure, mechanisms and motions	7
3.2	Sensors and actuators	8
3.3	Microcontroller programming	9
3.4	Simulation or prototyping/fabrication	10
4	Cost	12
5	Results and Discussion	13
6	System Overview	14
7	Conclusion and Further Developments	15
	References	16
A	Appendix A: Technical Specifications	17
B	Appendix B: Source Code	20

1 Introduction

The Warman Design and Build Competition is an yearly contest, in which undergraduate engineering students are challenged to create a fully autonomous robotic system to accomplish a specific engineering mission. The competition focuses on innovation, team work and implementation of mechatronics concepts like mechanical design, electronics and microcontroller programming. In the 2025 contest, the task was to create a robot capable of automatically fetching three types of balls a tennis ball, racquetball, and table tennis ball, from a 3 -by- 3 grid and placing them in a bunker on the other side of a pivoting seesaw. This whole process was to be done in 120 seconds and the robot must begin with a 400 mm x 400 mm x 400 mm cubic volume without any external manipulation after it is switched on.

When we began work on the project, our team discussed various design concepts to deal with the challenge. Among the early ideas was one featuring a rotating storage and lifting arms, and another having a hollow chassis, permanent ball grooves, and sweeping servo arms. We have evaluated these two designs with regards to simplicity, balance, and the likelihood of fabrication as well as mechanical stability on the seesaw and chose the second design. This was the basis of our final prototype, a four-wheeled robot powered by DC motors, with two servant powered arms to retain the balls, and a front collector that passively centres the balls in the correct position.

All motion and operating were- controlled by an Arduino Uno microcontroller powered by a time-based logic programmed sequence. We specifically did not design our system to have sensors or any kind of feedback, opting instead to calibrate the timings of the motors very accurately to get tasks done. This choice lowered the complexity of the system and the power requirements but created issues of accuracy and adaptability, especially when the field conditions vary.

The project was greatly refined after Week 6. A big challenge was getting the four DC motors (used in the navigation) and the two servos (used in the collection and release of the balls) to work in sync with each other. Their different power demand and torque characteristics would interfere with each other and cause instability at times, particularly when both systems were fed off the same battery. The solution we adopted to this was to adopt a dual power supply strategy, that is, one 12V Ni-MH battery pack to power the drive motors and a second 8xAA holder to power the servos. This change significantly increased the performance of the system and both the movement and collection actions could now be performed in parallel without voltage drop.

Although we had little access to the testing space, which was also commonly used by more than 50 students, and battery unreliability varied the timing of our movements, we managed to manipulate our delay sequences iteratively. The robot managed to finally grab the balls and go through the entire course, even climbing and balancing on the seesaw. In final demonstrations, though the tennis and racquet balls did not completely release (did not have enough forward momentum), the overall system worked reliably and satisfied the competition requirements.

This report details our design and development cycle as we brainstormed to our final testing. It describes our work on mechanical structure, motor and servo integration, code development, problems we had and things we learned. It also provides an understanding that even a single, un-augmented system, when designed with care, can suffice to face the challenging requirements of autonomy in the presence of realistic constraints.

2 Initial Design

To start the Warman project robot's development, we studied a few design approaches to find one that was both easy to build, dependable, and useful. The initial part of designing was to make sure the robot would gather balls and move across the seesaw in a secure way. Every design was checked to confirm it could work within the limits of the competition in terms of time, weight, and autonomous systems. The first concepts helped the team prepare for the prototype, which was made better and better through repeated testing and joint work.

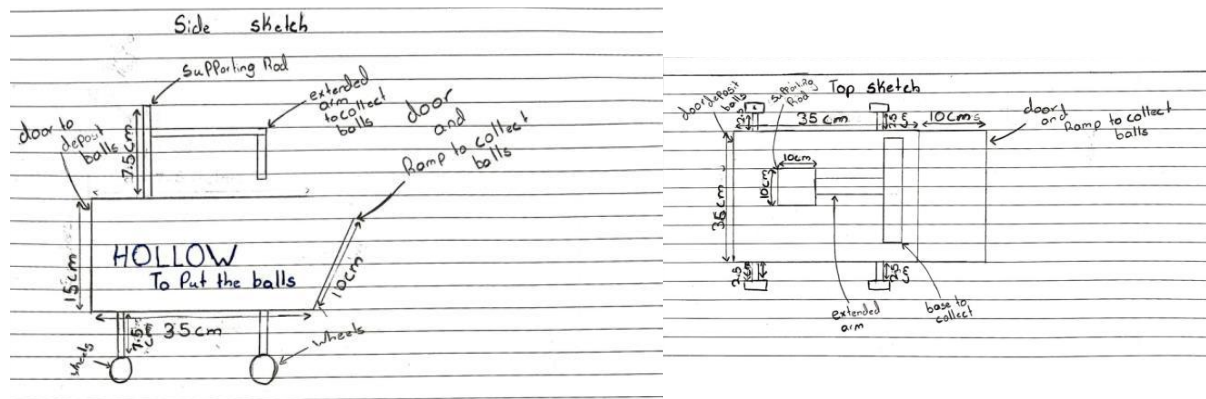


Figure 1.0 – Preliminary Design 1 Sketch.

The project started with two basis design concepts that were listed in our preliminary design report. Design 1 had a lightweight chassis and a pivoting arm, compared to Design 2, where you could find a storage box that could rotate and a lifting arm for picking up objects. Because of its effective, compact design, teamwork, and an easy shift in balance point, we decided on Design 1 as the best and simplest way to use the seesaw.

After seeing our designs in Weeks 4 and 5 not bring great results, we decided to focus on improvements such as the movement of the rover, collecting the balls, and controlling the weight on the seesaw. We chose to update our design.

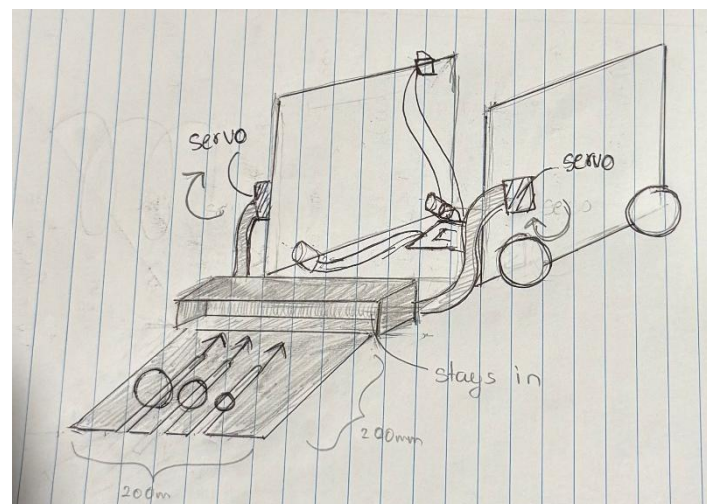


Figure 1.1 – Final Design Sketch.

The latest prototype shown in figure 1.1 comes with two side arms driven by servo motors and an operated front sweeper, which securely collects the ball for better practice and is powered by a separate battery set. The changed system makes collecting the ball more efficient and minimizes the chance of losing the balls when going over the seesaw. Now, the arms turn to hold the balls with the front sweeper that comes with a fixed box for the balls that is customized for various ball types.

2.1 Initial timelines

Table 01- Gantt chart before week 6 (Predicted results).

Task	W4	W5	W6	W7	W8	W9	W10	W11
Task 1: Design Sketching	○	○						
Task 2: Product Purchasing				○	○			
Task 3: Prototype Building					○	○		
Task 4: Sensor Testing						○		○
Task 5: Software Coding					○		○	○
Task 6: Testing and debugging						○	○	
Task 7: Final Enhancements							○	○

Table 02- Gantt chart after week 6 (Actual results).

Task	W4	W5	W6	W7	W8	W9	W10	W11
Task 1: Design Sketching	○	○						
Task 2: Product Purchasing			○	○	○	○		
Task 3: Prototype Building				○	○	○	○	○
Task 4: Sensor Testing								
Task 5: Software Coding			○	○	○	○	○	○
Task 6: Testing and debugging					○	○	○	○
Task 7: Final Enhancements							○	○

While the timeline prepared in Week 6 was meant to offer structure for the tasks, the actual project was carried out using a flexible and repeated approach. Although Task 1 (Design Sketching) went according to the plan in Weeks 4–5, some other tasks did not go as intended. The process of getting equipment for the project moved forward at the beginning of Week 6 due to the new design and lasted till Week 9 to anticipate the emergence of tougher hardware. Work on the project's prototype began a week ahead of plan and took longer than anticipated due to many changes with the servo arm system.

In the same way, the Software Coding work started two weeks early in Week 6 and finished in Week 11 as control integration was more detailed. Sensor Testing was not required as no sensors were used for the project. Finishing changes were made as early as Week 9, which promoted an easier merging of mechanical and software parts. Such changes indicate that the team continuously adapts, modifying timelines to match the real-time problems and chances for improvement during development.

It was especially hard to fine-tune the code that allowed the 4-DC motor rover to work properly with the 2-servo arm system. Aligning all the systems as they operated was essential, especially for picking up the ball and for the seesaw part, because the timing and movement of the robots were key. Coordinating the motors and servos so they did not interfere with one another took a lot of effort, especially since how they performed differed greatly from one another. Also, the initial problem was that the rover battery did not have enough power to control both the driving wheels and the robotic arms at the same moment. Because of this problem, the Power Crawler moved slowly and struggled to keep going on the incline. As a solution, we made sure the arm system and the movement system received power from separate and independent sources. As a result, the system ran more smoothly and each part of it performed well since it didn't have to share energy.

3 Final Design

Figure 2.0 below, demonstrates the marked CAD design of the robot used in the Warman project. On each side, there are two servo motors that guide the arms that move to collect the balls. Each servo gets its own battery pack to prevent drop in voltage when they work. The Arduino Uno inside the main chassis takes care of coordinating the robot's movements and arm. There are four DC motors using a separate battery pack to make the rover move. The front sweeper is made to carry different type of balls—tennis, racquet, and table tennis—and firmly place them in the fixed grooves that fit each ball on the chassis during transportation.

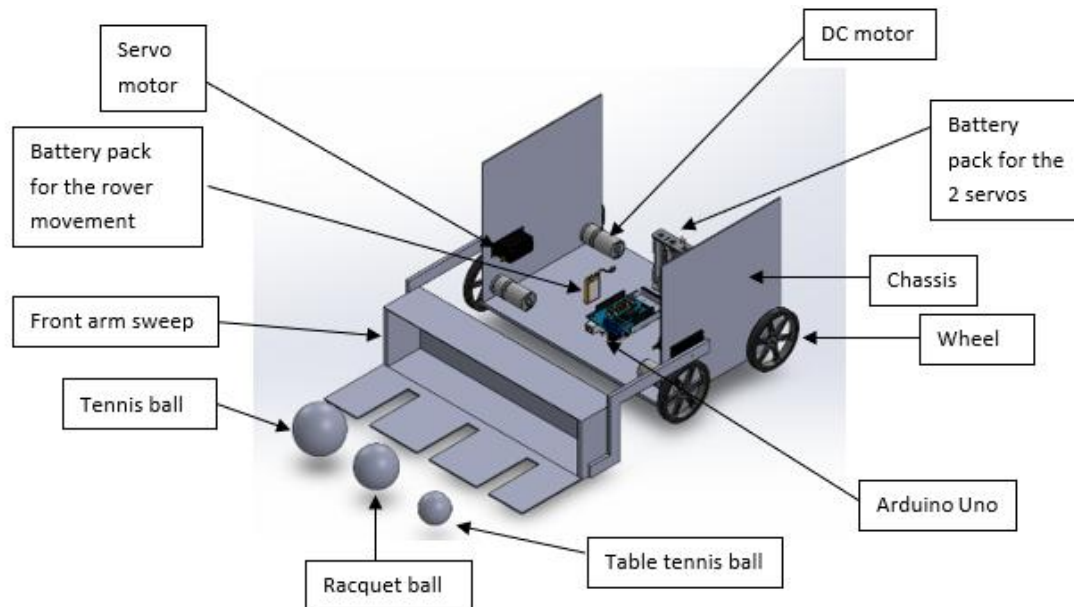


Figure 2.0 – Final Design CAD model.

3.1 Mechanical structure, Mechanisms and Motions

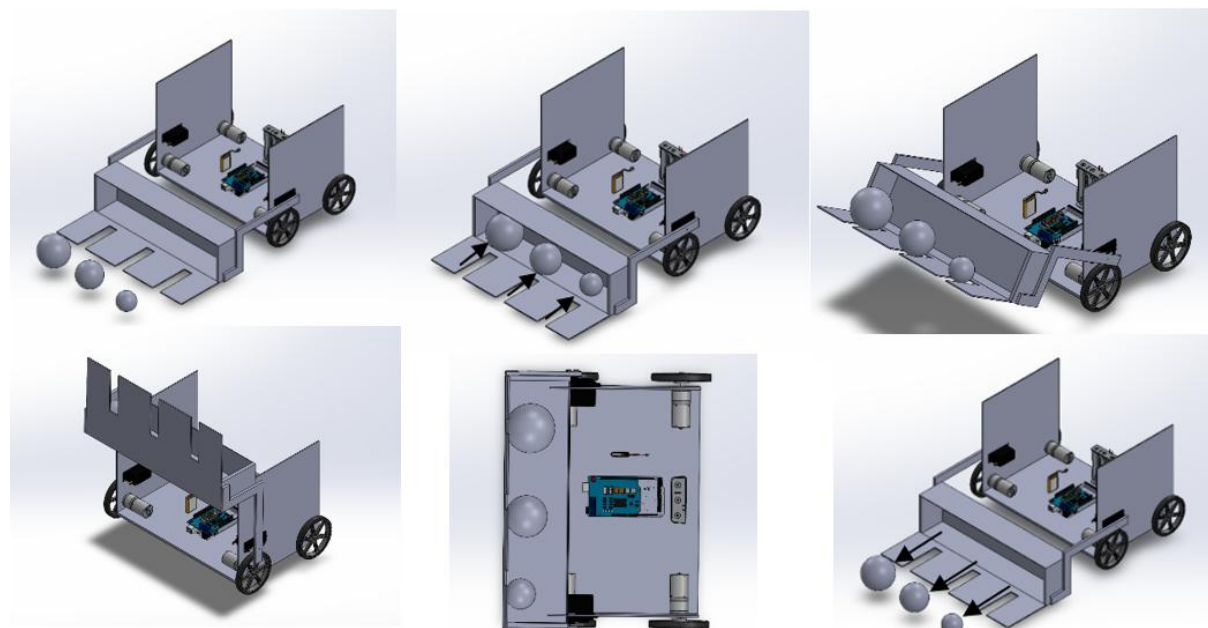


Figure 2.1 – Robot CAD design showing structural views and operational mechanisms.

The mechanical robustness of the robot was geared towards simplicity, robustness and consistent ball handling in a dynamic environment. The robot is made up of a rectangular chassis that rests on four DC motor powered wheels, side mounted servo arms, and a front collection system with custom ball grooves as shown in Figure 2.1. All these mechanical components are uniquely designed to fit the Warman competition specification, and they include safe transportation of the payload and stable movement over the seesaw.

The frame is a 3D printed chassis, designed to be lightweight and very stable. It encloses the Arduino Uno, motor shield and power distribution components in the middle to ensure balance. Structural stability was achieved by addition of two side panels to ensure that balls did not fall out during use. On the front panel, there are three slotted grooves each of which is calibrated to suit one of the three competition balls-tennis, racquet and table tennis- so that any of these balls rests firmly on its own track as the robot moves.

The robot gathers the payload with the two servo-driven arms, which are fixed on the contrary sides of the chassis. After the robot has driven into the area of the 3x3 grid, the arms swing inward, to hold the balls against the grooves, immobilizing them. This design obviates the requirement of any lifting or grabbing mechanism which makes the design mechanically simpler and more reliable in motion.

To place the balls into the bunker, the robot goes into the deposit zone and disengages the servos with a '.detach()' command in the code, resulting in the arms sliding in and dropping the payload. This is followed by quick forward motion which only gives sufficient energy to cause the balls to roll out of the front of the robot. The style does not involve any throwing action, which is safe about competition safety regulations, as well as provides soft and controlled placing of the balls.

The robot is driven with 4 12V DC gear motors in a differential drive configuration. Reverse and forward movements are accomplished by reversing polarity and turning is accomplished by driving the wheels of one side in opposite direction. This configuration enabled the robot to follow the grid, turn efficiently and traverse the seesaw effectively; this has been done without compromising the mechanical symmetry and balance of the robot.

All in all, the mechanical structure was constructed to accommodate both the physical and functional needs of the competition. It enabled the robot to achieve most of its goals automatically and in a stable manner, with controlled payload all through its course.

3.2 Sensors and Actuators

We have an autonomous system that does not use any sensors and is based upon pre-programmed actuation sequences. Motion of the robot is accomplished using a combination of DC motors and controlling the arm mechanism using servo motors, both chosen considering the required torque, Voltage compatibility and reliability.

Four 12V 70RPM reversible DC gearhead motors are used to supply the rover movement with adequate torque and steady speed needed to climb ramps and negotiate through the course [3]. Directional and speed control is controlled using the Arduino Motor Shield Rev3 that allows pulse-width modulation and easy connection with the Arduino Uno Rev3 microcontroller, both of which are obtained from Jaycar [1][2]. With this arrangement the robot can be controlled by completely autonomous, time-driven instructions with extremely simple circuits.

In the case of the ball collection mechanism, we have taken two Hitec HS-422 servo motors which can provide up to 3.9 kg-cm of torque at 6V. These servos were selected because of their accurate controlled rotation, sturdy construction, and dual bearings that allow smooth repeated motion without jitter that is so critical to positive arm movement [4].

We used the dual power supply system to achieve the same performance of all motors. A 12V Ni-MH 1800mAh battery pack is used to power the DC motors, and it provides enough current to facilitate high-load operations, like climbing ramps [5]. The two servos are supplied independently through an 8 x AA battery holder, keeping the servo voltage independently, and minimizing the chance of momentary voltage dips compromising performance [6].

Through tailored actuator-task pairing and isolation of power systems, the robot can exactly and effectively accomplish any action without use of real-time sensory feedback.

3.3 Microcontroller Programming

The Arduino IDE located on the robot is programmed in Arduino IDE.exe software, and the logic is carried out sequentially by a chain of timed movements and servo functions. All movement is planned out with pre-determined time-delays and motor-commands, so calibration is important during development.

Signal processing and logic

There is no sensor or external feedback of any kind that the robot is depending on, thus all control logic is implemented in terms of absolute time intervals and hard-coded motor instructions. This rendered proper timing calibration to be of uttermost importance as concerns the success of the system. All the movements, driving forward, in reverse, turning and using the arms, had to be timed accurately to match the physical distances and functional demands on the competition field.

To do this we had to go through a lengthy trial and error process where we would have to repeatedly run the robot on the test track adjusting each of the delay values. To give an example, we had to measure the precise distance that the robot needed to travel forward to enter the ball area and not go too far, the same with reversing, how much time it needed to turn to look in the right direction.

Such cycle of test, measure, adjust enabled us to construct schedule of operations which satisfied the constraints of the challenge with the least amount of complexity. It also made up the absence of real time sensory correction and in the end our robot was able to finish the entire course with just well programmed motion sequences.

Program Flow (Pseudocode method)

Table 03 – Program flow

Time (s)	Action	Description
0.00 – 1.30	Move forward	Initiate all 4 motors to move forward.
1.30	Stop	Motors stop.
1.30 – 2.35	Move servos to 90°	Arm positions to pick the balls using leftServo.write(0) and rightServo.write(90).
2.35 – 3.45	Pause	Allow arms to stabilize and fully secure the balls collected.
3.45 – 4.35	Move backwards	Reverse slightly to get ready to turn the back of the rover towards the seesaw.
4.35 – 5.35	Pause	Pause before turn.
5.35 – 6.10	Turn right	Left two motors go forward, right two motors go reverse to perform a right pivot turn.
6.10 – 7.10	Pause	Stabilize after turn.

7.10 – 7.30	Move forward	Short forward movement to create momentum for the incline run.
7.30 – 8.30	Pause	Pause after forward movement.
8.30 – 13.80	Move backwards	Crossing the seesaw in reverse.
13.80 – 18.80	Pause	Pause for the seesaw to go down.
18.80 – 19.80	Move backwards	Reverse to get out of the seesaw.
19.80 – 20.65	Turn right	Turning right to for the arms to face the deposit zone.
20.65 – 20.75	Move forward	Moving towards the deposit zone.
20.75 – 23.75	Pause	Pause to stabilize.
23.75 – 24.75	Move servos to original position	Moving the arm down to drop the balls.
24.75 – 25.15	Move forward	Moving forward to create momentum for the balls to fully come out of the arm.
25.15+	Full stop	Program over.

Code Execution

All the code is written within the 'setup ()' function, indicating that it will be performed once, beginning to end, non-repeating and non-conditional. The 'loop ()' is left empty after the sequence execution to avoid repeated actions. This architecture is appropriate to the one-off independent Warman competition task.

The entire source code is provided in the appendix to refer and examine.

Challenges

During the development of time-based control system, there were several practical issues, which complicate the process of calibration and debugging considerably. Major among the problems was the inadequate access to the testing area. There were almost 50 students on the same track as us so we would wait hours just to get a few minutes to do one test. This hindered the rapid iteration and sometimes a small change in the code would take several days and we could not test the change in real time. Such inconsistency introduced an element of unpredictability into our delay-based programming model and required us to consistently recharge or otherwise replace batteries between trials to achieve consistency.

Voltage drops and battery inconsistency was another serious problem that directly affected the accuracy of movement. When the battery charge became low over time the speed and torque of the DC motors changed which caused the robot to overshoot or undershoot its desired position- with the same timing logic.

Collectively these issues demonstrated the shortcoming of pure timed motion without sensory feedback and the value of system stability and repeatability operating in a shared, competitive space.

3.4 Simulation or Prototyping/Fabrication

Our Warman robot was developed under a prototyping and fabrication-based approach, so we were able to build and test our mechanical structure physically. Figure 3.0 shows that the initial steps of the fabrication process were to assemble the chassis, wire up the electronic components, integrate the Arduino and motor shield, and mount the servo mechanisms to the sweeping arms. The accuracy of the components placement was

critical during this stage particularly the weight balancing and non-interference of the wheels and arms movements. A lot of the work was done in the university workshop where the team assisted each other with fitting the ball grooves, testing the arm rotations and adjusting the internal layouts of the power delivery and accessibility.

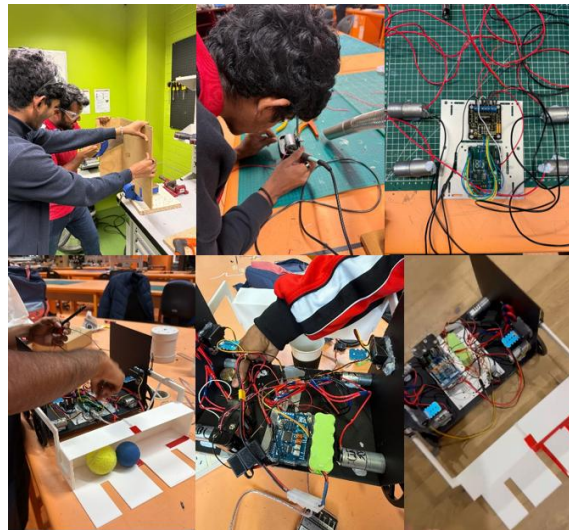


Figure 3.0– Fabrication process and internal assembly.

After fabrication, the next stage was full-scale testing and demonstration as shown in Figure 3.1. This stage involved setting the robot at the initial area, picking the 3 balls of various sizes (tennis, racquet, and table tennis), and moving over the seesaw to the deposition area. The mechanical integrity and the functionality of the code were confirmed by the performance of the robot. These runs provided observations that would result in final adjustments in servo angles and motor timing to provide smooth transition over obstacles and proper placement of the ball. This phase was successful; however, the tennis ball and the racquet ball did not go out of the arm sweep. This was due to lack of momentum for the balls to go out and we had only two demos to calculate the timing. Other than that, it was established that the robot could finish the entire course within the competition requirements.

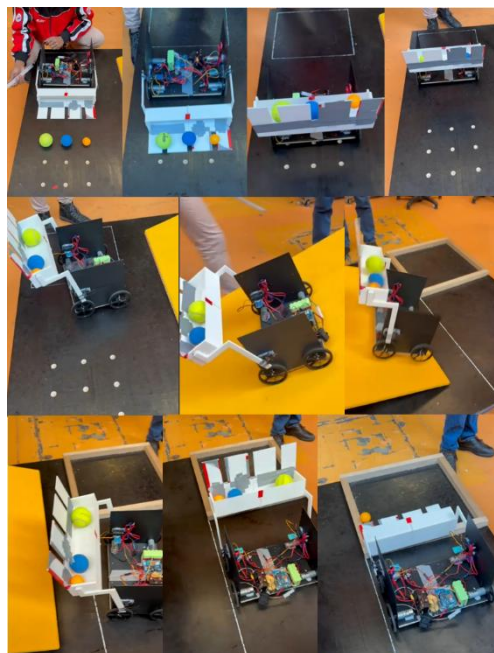


Figure 3.1 – Project demonstration.

4 Cost

Table 04- Cost Divide per member analysis

Materials	Cost (\$)
3-D print	113.6
Hot glue sticks	7.2
Wires	7.95
Switch	4.95
Battery holder	3.4
Batteries	31
Total	168.1
Per person	33.62

Materials used from the University

- 1 Wooden Sheet
- 1 Arduino Uno
- 1 Motor shield
- 2 Servo motors
- 4 DC motors
- 1 Battery pack
- 4 Wheels
- 4 Wheel mounts (Adapters)
- 1 Set of wires
- 4 Weights

As it appears from the materials cost table, the total cost of the project was \$168.10. Because the university gave us access to important resources, we were able to lower our expenses a lot. All the university gave us was the Arduino Uno, motor shield, various motors, a battery pack, and wheels, which otherwise would have greatly raised our expenses. Thanks to the resources, we probably cut more than \$500 from the expenses. All the other items, for example, the 3D prints, wires, batteries, switches, adhesives, and electrical components, were bought at Jaycar Electronics and a printing studio. Using Jaycar's store wasn't a problem, since we found a huge variety of electronic parts available locally and our build didn't get delayed. The university gave us useful materials and local access to supplies allowed us to continue under budget, no matter the quality.

5 Results and Discussion

The completed system was able to autonomously operate successfully through a set of timed tasks, making the system rivet the objectives of the Warman competition. That robot managed to overcome the major obstacles of ball collection, navigation, seesaw crossing, and final delivery, all without any sensors, just pre-programmed motor control and timed delay sequences.

Task 01: Ball collection

The system lacks any ball detecting sensors, but the timing-based system, coupled with the fixed position of the balls on the field allowed the robot to drive to the correct position to pick up the ball. These two Hitec HS-422 servo motors were rotated inwards to catch the tennis ball, racquet ball and table tennis ball into their corresponding grooves. The front plate was 3D-printed with these grooves and that aligned each ball perfectly in place when moving.

Task 02: Controlled movement

The four 12V DC gear motors which were operated through Arduino Motor Shield allowed direction control by programming the PWM signals. The reverse and forward movements and turning movements were obtained by reversing the polarity of each pair of motors. Direction changes were also adjusted accurately to align the paths properly and were tuned by a lot of trial and error to suit the varying frictions of the surfaces.

Task 03 – Travelling across the seesaw

A serious mechanical problem was crossing the seesaw. There was no tilt sensors employed, so the robot drove to the seesaw with a fixed speed and counted on the steady battery power to keep itself balanced in the crossing. The two-battery system contributed to counteract the effects of volt drop thereby enabling the DC motors to run at full power. Experimental results indicated that the robot was capable of stable climbing, balancing and descending without any instability or shifting of the payload.

Task 04- Unloading of the balls

In the deposit area the arms were switched off by unplugging the servo signal '(with .detach())', and the balls were dropped easily into the storage bunker. This was accompanied by a brief forward thrust to assure full delivery. The tennis ball and the racquet ball, however, did not completely get out of the arm sweep in the last release. The cause of this problem was identified to be a lack of energy to propel the heavier balls out of the containment grooves considering that the release was purely depends on gravity and a small amount of forward motion. Also, the team was given two demo attempts to adjust timing and maximize arm positioning, which did not provide much room to refine. This was a slight disappointment, but the table tennis ball was finally released and the system proved that it was able to go through the whole course according to the requirements in the competition.

System Overview

The prototype uses a deliberately simple architecture: timed Arduino control, differential drive, servo-assisted ball retention, and isolated power for the drive and handling systems.

01

CONTROL

An Arduino Uno executes one calibrated sequence of forward, reverse, turning, pickup, crossing, and release actions. The loop remains empty after completion.

02

DRIVE

Four 12 V geared DC motors use differential steering. Reversing wheel polarity enables forward motion, reverse motion, and pivot turns.

03

BALL HANDLING

Two servo-driven arms retain the balls against dedicated chassis grooves during movement. Servo detachment releases the payload at the destination.

04

POWER

A 12 V Ni-MH pack supplies the drive motors, while a separate 8 x AA holder powers the servos. Isolation reduces voltage drop during combined operation.

OPERATING SEQUENCE

Approach grid > secure balls > reverse and turn > cross seesaw > align with destination > release payload

7 Conclusion and Further Developments

The project was greatly refined after Week 6. A big challenge was getting the four DC motors (used in the navigation) and the two servos (used in the collection and release of the balls) to work in sync with each other. Their different power demand and torque characteristics would interfere with each other and cause instability at times, particularly when both systems were fed off the same battery. The solution we adopted to this was to adopt a dual power supply strategy, that is, one 12V Ni-MH battery pack to power the drive motors and a second 8xAA holder to power the servos. This change significantly increased the performance of the system and both the movement and collection actions could now be performed in parallel without voltage drop.

Although we had little access to the testing space, which was also commonly used by more than 50 students, and battery unreliability varied the timing of our movements, we managed to manipulate our delay sequences iteratively. The robot managed to finally grab the balls and go through the entire course, even climbing and balancing on the seesaw. In final demonstrations, though the tennis and racquet balls did not completely release (did not have enough forward momentum), the overall system worked reliably and satisfied the competition requirements.

This report details our design and development cycle as we brainstormed to our final testing. It describes our work on mechanical structure, motor and servo integration, code development, problems we had and things we learned. It also provides an understanding that even a single, un-augmented system, when designed with care, can suffice to face the challenging requirements of autonomy in the presence of realistic constraints.

Not every detail of the design accomplished our objectives though. In end of demonstration tests, the tennis and racquet balls were not completely out of the collector at the delivery point. This was seen as a weakness because it lacked adequate forward push as well as power in the arm release mechanism which was purely dependent on gravity and a small amount of forward movement. Moreover, restricted accessibility to the test arena along with the fluctuations in battery voltage affected the consistency of timing that made the calibration more challenging and confined the modification in real-time.

These problems have been learning difficulties. We got to know that even though sensor-free system is easier and more effective in some cases, they are also less able to adjust to the variability. Future enhancements would incorporate simple sensors- IR or limit switches to provide real time positional feedback and reworking the ball release mechanism to provide some mechanical assist (e.g. a tilting floor or triggering gate). Also, specific test time and addition of voltage regulation components may help to increase repeatability and competition conditions performance.

To conclude, our group was able to develop a mechanically stable, electrically solid, and functionally independent system that still met most of the competition requirements. The robot may become a more adaptive and competitive system in the next generations with minor improvements and the ability to sense.

References

- [1] "Arduino UNO Rev3 Arduino Development Board," *Jaycar*. <https://www.jaycar.com.au/arduino-uno-rev3-arduino-development-board/p/XC9202>
- [2] "Arduino Motor Shield Rev3," *Jaycar*. <https://www.jaycar.com.au/arduino-motor-shield-rev3/p/XC9207>
- [3] "70RPM 12VDC Reversible Gearhead Motor," *Jaycar*. <https://www.jaycar.com.au/70rpm-12vdc-reversible-gearhead-motor/p/YG2732>
- [4] "HiTEC HS-422 Indirect Drive Dual Oilite Bearings," *Metro Hobbies*. https://www.metrohobbies.com.au/products/hitec-hs-422-indirect-drive-dual-oilite-bearings?variant=12970286514229&country=AU&cy=AUD&utm_medium=product_sync&utm_source=google&utm_content=sag_organic&utm_campaign=sag_organic&gad_source=1&gad_campaignid=20853545442&gbraid=0AAAADz0wOnhEAuzslaBabNqZPLbZNWak&gclid=CjwKCAjw9anCBhAWEiwAqBJ-c5TWY4a9SKjuQyJ1EvvoQQhHgoZX-C7TEeiXHwPa9ZUu-SjWY6BJTxoCVrYQAvD_BwE&gclidsrc=aw.ds
- [5] "Ni-MH AA 12V 1800mAh Big White Plug Battery Pack-10 Pcs a Pack (Dual Level) ," *Lightmalls*. <https://au.lightmalls.com/ni-mh-aa-12v-1800mah-big-white-plug-battery-pack-10-pcs-a-pack-dual-level>
- [6] "8 X AA 2 Rows of 4 Square Battery Holder," *Jaycar*. https://www.jaycar.com.au/8-x-aa-2-rows-of-4-square-battery-holder/p/PH9209?gad_source=1&gad_campaignid=21866462775&gbraid=0AAAAAD0dvLZEbY0MgSR3lAgGbVgtSR8vm&gclid=CjwKCAjw9anCBhAWEiwAqBJ-c91wgvXvIIVB9e-c7wxuaTFZbBA1fPLOK-Drm5xtblFHj2fHwQD_2hoC94EQAvD_BwE

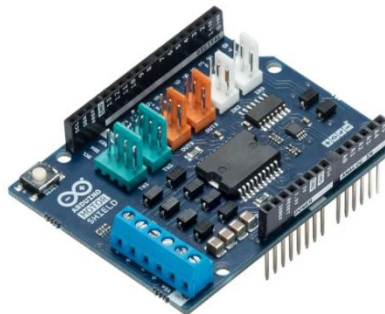
Appendix A: Technical Specifications

7.1 Arduino Uno r3 Specifications



Specification	Value
Microcontroller	ATmega328P
Program Memory	32.0 KB
Product Length	68.6 mm
Product Width	53.4 mm
Product Weight	25.0 g
Packaged Length	8.2 cm
Packaged Width	6.2 cm
Packaged Height	2.7 cm

7.2 Motor Driver (L298P) Specifications



Specification	Value
Sensor IC	L298P
Product Length	68.6 mm
Product Width	53.4 mm
Packaged Length	8.4 cm
Packaged Width	6.2 cm
Packaged Height	3.9 cm

7.3 DC Gear Motor Specifications



Specification	Value
Motor Nominal Voltage	12.0 V
No Load Current	30.0 mA
Full Load Current	390.0 mA
Full Load Torque	2.1 kg·cm
No Load RPM	70.0 RPM
Shaft Diameter	4.0 mm
Shaft Length	8.8 mm
Max Efficiency Current	110.0 mA
Max Efficiency	49.3 %
Max Efficiency RPM	59.0 RPM
Max Efficiency Power	1.3 W
Maximum Efficiency Torque	480.0 g·cm
Motor Type	Gearhead
DC Voltage Range	4.5V, 18.0V
Item Connection	Solder Tags
Pre-assembled Gear	True
Gear Ratio	82:1
Adjustable Gear	False
Gearbox Type	Planetary/Orbital
Packaged Length	7.7 cm
Packaged Width	2.9 cm
Packaged Height	2.8 cm

7.4 Hitec HS-422 Servo Motor Specifications



Specification	Value
Size Category	Standard
Type	Analog
Application	20–60 Size Aircraft, 30 Size Helicopters; Cars, Trucks, Boats
Torque	45.8 oz-in (3.3 kg·cm) @ 4.8V; 56.9 oz-in (4.1 kg·cm) @ 6.0V
Speed	0.21 sec/60° @ 4.8V; 0.16 sec/60° @ 6.0V
Length	1.59 in (19.8 mm)
Width	0.77 in (40.6 mm)
Height	1.44 in (36.6 mm)
Weight	1.60 oz (45.5 g)
Bushing or bearing	Bushing
Bearing	Dual
Motor Type	3 Pole
Connector Type	Universal
Gear Type	Nylon
Gear Material	Resin
Current Draw (Idle)	8 mA
Voltage Range	4.8V to 6.0V
Deadband	8 µs

7.5 Ni-MH AA Battery Pack Specifications



Specification	Value
Battery Type	Ni-MH
Eco-Friendly	Yes, with no memory effect
Voltage	12V
Size	AA
Plug Style	Big white plug
Capacity	1800 mAh
Quantity per Pack	10 pcs
Cable Polarity	Red = “+”, Black = “-”
Usage	Toy cars, model planes, ships, electronics
Caution	Do not short circuit or mix with others
Package Weight	0.3 kg
Package Dimensions	8 × 4 × 6 cm

7.6 8 x AA Battery Holder Specifications



Specification	Value
Type of Cell	AA
Number of Cells	8 pcs
Cell Configuration	Square
Output Connection	Flyleads
Length	58.0 mm
Width	60.0 mm
Height	30.0 mm
Cable Length	150.0 mm
Packaged Dimensions	6.0 cm (L) × 6.5 cm (W) × 2.5 cm (H)

Appendix B: Source Code

<pre> #include <Servo.h> // Servo setup (Arm) Servo leftServo; Servo rightServo; // Motor control (Rover) const int rightPolarity = 12; const int leftPolarity = 13; const int rightBrake = 9; const int leftBrake = 8; const int rightSpeed = 3; const int leftSpeed = 11; void setup() { pinMode(rightPolarity, OUTPUT); pinMode(leftPolarity, OUTPUT); pinMode(rightBrake, OUTPUT); pinMode(leftBrake, OUTPUT); // t = 0.00–1.30: Forward movement digitalWrite(rightPolarity, HIGH); digitalWrite(leftPolarity, HIGH); digitalWrite(rightBrake, LOW); digitalWrite(leftBrake, LOW); analogWrite(rightSpeed, 200); analogWrite(leftSpeed, 240); delay(1300); // t = 1.30: Stop forward movement analogWrite(rightSpeed, 0); analogWrite(leftSpeed, 0); digitalWrite(rightBrake, HIGH); digitalWrite(leftBrake, HIGH); // t = 1.30–2.35: Arm moves to 90 degrees leftServo.attach(8); rightServo.attach(9); leftServo.write(0); rightServo.write(90); delay(1050); // t = 2.35–3.45: Pause delay(1100); // t = 3.45–4.35: Reverse for 0.9s digitalWrite(rightBrake, LOW); digitalWrite(leftBrake, LOW); digitalWrite(rightPolarity, LOW); digitalWrite(leftPolarity, LOW); analogWrite(rightSpeed, 220); analogWrite(leftSpeed, 245); delay(900); // t = 4.35–5.35: Pause after reverse analogWrite(rightSpeed, 0); analogWrite(leftSpeed, 0); digitalWrite(rightBrake, HIGH); </pre>	<pre> analogWrite(leftSpeed, 255); delay(750); // t = 6.10–7.10: Pause after turn analogWrite(rightSpeed, 0); analogWrite(leftSpeed, 0); digitalWrite(rightBrake, HIGH); digitalWrite(leftBrake, HIGH); delay(1000); // t = 7.10–7.30: Forward for 0.2s digitalWrite(rightBrake, LOW); digitalWrite(leftBrake, LOW); digitalWrite(rightPolarity, HIGH); digitalWrite(leftPolarity, HIGH); analogWrite(rightSpeed, 200); analogWrite(leftSpeed, 240); delay(200); // t = 7.30–8.30: Pause analogWrite(rightSpeed, 0); analogWrite(leftSpeed, 0); digitalWrite(rightBrake, HIGH); digitalWrite(leftBrake, HIGH); delay(1000); // t = 8.30–13.80: Reverse (5.5s) right:240, left:255 digitalWrite(rightBrake, LOW); digitalWrite(leftBrake, LOW); digitalWrite(rightPolarity, LOW); digitalWrite(leftPolarity, LOW); analogWrite(rightSpeed, 240); analogWrite(leftSpeed, 255); delay(5500); // t = 13.80–18.80: Pause (5s) analogWrite(rightSpeed, 0); analogWrite(leftSpeed, 0); digitalWrite(rightBrake, HIGH); digitalWrite(leftBrake, HIGH); delay(5000); // t = 18.80–19.80: Reverse again (1.0s) digitalWrite(rightBrake, LOW); digitalWrite(leftBrake, LOW); digitalWrite(rightPolarity, LOW); digitalWrite(leftPolarity, LOW); analogWrite(rightSpeed, 240); analogWrite(leftSpeed, 255); delay(1000); // t = 19.80–20.65: Turn RIGHT again for 0.85s digitalWrite(rightBrake, LOW); digitalWrite(leftBrake, LOW); digitalWrite(rightPolarity, LOW); digitalWrite(leftPolarity, HIGH); analogWrite(rightSpeed, 230); analogWrite(leftSpeed, 255); delay(850); </pre>	<pre> delay(850); // t = 20.65–20.75: Forward for 0.1s digitalWrite(rightPolarity, HIGH); digitalWrite(leftPolarity, HIGH); analogWrite(rightSpeed, 200); analogWrite(leftSpeed, 240); delay(100); // t = 20.75–23.75: Final pause (3s) analogWrite(rightSpeed, 0); analogWrite(leftSpeed, 0); digitalWrite(rightBrake, HIGH); digitalWrite(leftBrake, HIGH); delay(3000); // t = 23.75: Final arm release leftServo.detach(); rightServo.detach(); delay(1000); // 1s pause after final arm release // Final forward movement for 0.4s digitalWrite(rightBrake, LOW); digitalWrite(leftBrake, LOW); digitalWrite(rightPolarity, HIGH); digitalWrite(leftPolarity, HIGH); analogWrite(rightSpeed, 200); analogWrite(leftSpeed, 240); delay(400); // Full stop analogWrite(rightSpeed, 0); analogWrite(leftSpeed, 0); digitalWrite(rightBrake, HIGH); digitalWrite(leftBrake, HIGH); } void loop() { // Do nothing } </pre>
--	---	--